# Integrasi Ekosistem Super Dev Builders — GitHub, GitHub Desktop, Vercel, Supabase

**Kajian kelayakan dan rekomendasi arsitektur**
Tanggal: 7 Oktober 2026 · Disusun oleh: Hermes Anantara v3
Sumber kebenaran: `~/super-dev-builders/docs/00_MASTER_BLUEPRINT_V2.2.html` + README (7 aturan non-negosiasi)

## Ringkasan untuk yang tidak sempat membaca semuanya

Pertanyaan Anda: apakah ekosistem Super Dev Builders (SDB) perlu disambungkan ke **GitHub**, **GitHub Desktop**, **Vercel Free**, dan **Supabase**?

Jawaban singkat, dengan skor kebutuhan:

| Layanan | Perlu? | Untuk apa | Urutan |
|---|---|---|---|
| **GitHub** | **Ya — wajib, sekarang** | Rumah repositori: version control, kolaborasi, GitHub Actions (CI), GitHub Pages | **1 (hari ini)** |
| **GitHub Desktop** | **Tidak perlu** (untuk mesin ini) | Aplikasi grafis untuk commit/push. Di sini kerja dilakukan dari terminal & agen, dan sudah ada `git` + `gh` | — |
| **Vercel Free** | **Ya, tapi nanti (fase demo)** | Menghosting hasil karya (situs/aplikasi jadi) supaya klien bisa melihatnya lewat satu tautan | **2 (setelah repo beres)** |
| **Supabase** | **Belum perlu** (hanya bila ada aplikasi yang butuh database) | Database + login pengguna + API otomatis untuk aplikasi yang dibangun | **3 (per aplikasi, bukan sekarang)** |

Analogi sederhana:

> Bayangkan sebuah bengkel. **GitHub** = lemari arsip bengkel (menyimpan setiap versi cetak biru, dan bisa "mesin otomatis" merakit). **GitHub Desktop** = satu lagi pintu masuk ke lemari itu; karena kita sudah masuk lewat jalur lain, pintu tambahan belum perlu. **Vercel** =

> ruang pamer di depan toko, tempat pelanggan melihat barang jadi. **Supabase** = gudang + petugas administrasi, hanya dibutuhkan kalau barang yang dijual memang pakai gudang.

---

## Daftar Isi

---

## Bab 1 — Pengantar: apa yang sebenarnya ditanyakan

Pertanyaannya bukan sekadar "bisa disambung atau tidak" (semua bisa). Pertanyaan yang lebih tepat: **sambungan mana yang menambah kecepatan menuju target, dan sambungan mana yang menambah risiko tanpa manfaat** — terutama karena SDB punya 7 aturan non-negosiasi yang ketat soal izin, bukti, dan rahasia.

Tiga hal yang jadi ukuran dalam dokumen ini:

1. **Manfaat nyata** — apakah mempercepat sesuatu yang sekarang harus dikerjakan manual?
2. **Risiko baru** — apakah membuka pintu untuk data klien, rahasia (API key), atau urusan hukum yang sebelumnya tidak ada?
3. **Aturan SDB** — apakah tetap patuh pada 7 aturan repo (tanpa izin tertulis tidak ada deep scan; tanpa approval manusia tidak ada deploy produksi; tanpa bukti segar tidak ada klaim "lulus")?

---

## Bab 2 — Kondisi sekarang (hasil pemeriksaan, bukan perkiraan)

Semua angka di bawah ini hasil pemeriksaan langsung di mesin ini pada 7 Oktober 2026.

### 2.1 Perkakas yang sudah tersedia

| Perkakas | Status | Catatan |
|---|---|---|
| git | **2.43.0** | terpasang |
| gh (GitHub CLI) | **2.102.0** | terpasang, **belum login** |
| Docker | **29.8.2** | terpasang — bisa menjalankan Supabase lokal |

| Perkakas | Status | Catatan |
|---|---|---|
| Node.js | **26.7.0** | terpasang |
| npm | **11.19.0** | terpasang |
| Google Chrome | ada | untuk test UI dan cetak PDF |
| Playwright + Chromium | ada (cache) | untuk uji otomatis halaman |
| GPU NVIDIA | **tidak ada** | relevan: Supabase lokal (Docker) jalan di CPU — cukup |

### 2.2 Repositori SDB

| Hal | Nilai |
|---|---|
| Lokasi | `~/super-dev-builders` |
| Remote GitHub | **belum ada** (belum tersambung ke mana pun) |
| Kualitas | 140 test PASS, ruff 0 temuan, `make check` hijau |
| Drift skill | 106/106 OK (skill di repo = skill terpasang) |
| Skill inti | 37 skill kustom, 7 profil agen |

**Kesimpulan Bab 2**: perkakas sudah lengkap. Yang belum ada hanyalah **sambungan** — dan itu yang paling murah serta paling besar manfaatnya hari ini.

---

## Bab 3 — Prinsip tetap: tidak ada yang tepercaya secara bawaan

Ini penting sebelum membahas layanan apa pun, karena layanan cloud = pihak ketiga.

SDB memegang prinsip **#0: semua target, repositori, tautan, dependency, skill, dan alat diperlakukan "untrusted" (tidak tepercaya) sampai lulus gerbang pemeriksaan.**

Konsekuensi praktis untuk sambungan cloud:

| Prinsip | Artinya dalam praktik integrasi |
|---|---|
| Rahasia tidak pernah masuk repo/evidence | Kunci Vercel/Supabase/GitHub disimpan sebagai *environment variable* (variabel lingkungan), bukan di file yang di-commit |
| Deploy produksi = keputusan manusia | Vercel **diizinkan untuk demo/pratinjau**, tetapi "naik ke produksi" tetap butuh persetujuan Anda |
| Tidak ada klaim tanpa bukti segar | Setiap integrasi harus punya bukti: skrip uji, keluaran nyata, checksum |
| Satu nama satu pemilik | Integrasi per layanan dibuat sebagai **divisi/berkas terpisah**, tidak menumpuk di satu file besar |
| Data pribadi (PII) tidak masuk konteks LLM | Untuk demo ke klien, **gunakan data palsu/dummy** — jangan data warga asli |

> Analogi: menyambungkan listrik ke rumah baru tidak berarti semua kabel boleh dibiarkan terbuka. Sambungan boleh, tapi harus lewat sekring (gerbang izin) dan tidak boleh ada kabel telanjang (rahasia) yang menyentuh lantai (repo publik).

---

## Bab 4 — GitHub: jawabannya "ya, sekarang", dan mengapa

### 4.1 Tiga manfaat langsung

1. **Version control yang aman.** Sekarang seluruh riwayat hanya ada di mesin ini (git lokal). Bila disk rusak, riwayat hilang. GitHub = salinan aman di tempat lain.
2. **GitHub Actions = mesin pemeriksa otomatis.** Test, ruff, `registry_lint`, `doctor` bisa dijalankan otomatis setiap kali ada perubahan. Ini memindahkan budaya "bukti segar" dari disiplin manual menjadi check otomatis.
3. **GitHub Pages = ruang pamer gratis.** Halaman statis bisa langsung dipublikasikan (misalnya dokumentasi SDB atau hasil karya) tanpa biaya.

### 4.2 Yang harus diperhatikan

| Risiko | Mitigasi |
|---|---|
| Repo berisi rahasia (kunci API) → terbaca publik | Sebelum push pertama: pindai rahasia (`gitleaks`/pola), pastikan `.env` ada di `.gitignore`, dan **rotasi kunci apa pun yang pernah bocor** |
| Memori saya mencatat: kredensial nyata pernah muncul di riwayat commit | **JANGAN di-push sebelum rotasi kunci** — ini temuan yang sudah tercatat sebelumnya |
| Repo publik = orang lain bisa memakai karya kita | Mulai dari **privat**. Naikkan ke publik hanya untuk bagian yang memang mau dibagikan |
| Aturan #2 melarang bypass; repo publik memudahkan orang menyalahkan kita | Pasang `LICENSE` yang jelas + `SECURITY.md` (cara melapor) |

### 4.3 Rekomendasi konkret




### 4.4 GitHub Pages — untuk apa dan untuk siapa

| Cocok untuk | Kurang cocok untuk |
|---|---|
| Dokumentasi SDB (blueprint, ADR, panduan) | Aplikasi yang butuh server/API (Pages hanya statis) |
| Laporan audit yang boleh dibaca orang lain | Halaman dengan data klien asli |
| Showcase hasil karya versi statis | Aplikasi butuh login pengguna |

**Kesimpulan Bab 4**: GitHub bukan "boleh", tapi **seharusnya** — dan biaya nol untuk kebutuhan kita.

## Bab 5 — GitHub Desktop: jawabannya "tidak perlu" (di mesin ini)

**GitHub Desktop** adalah aplikasi grafis untuk commit, push, dan melihat perubahan lewat klik. Ia berguna kalau pekerjaannya dilakukan manusia yang tidak nyaman dengan terminal.

Di sini situasinya berbeda:

| Fakta | Konsekuensi |
|---|---|
| `git` 2.43.0 dan `gh` 2.102.0 sudah ada | Operasi commit/push/branch sudah bisa, dan otomatis |
| Pekerjaan utama dilakukan **agen** (Hermes, subagent), bukan klik manual | Antarmuka grafis justru tidak bisa dijalankan agen |
| Sudah ada jalur "mesin": GitHub Actions + CLI | Lebih cepat, terekam, bisa diaudit |

Analoginya: kalau dapur sudah punya pintu otomatis (CLI + agen), menambah pintu manual (Desktop) tidak salah — hanya **tidak menambah kecepatan**, malah menambah satu hal lagi yang harus dijaga.

**Kapan GitHub Desktop jadi masuk akal?** Bila *Anda* sendiri ingin sesekali melihat perubahan (diff) secara visual dan memberi tanda centang (review) manual. Itu **peran Anda sebagai pemilik risiko** — dan itu tetap bisa dilakukan lewat GitHub di browser (tab "Pull requests"), tanpa memasang aplikasi.

**Rekomendasi**: **lewati**. Gunakan halaman web GitHub untuk tinjauan manual. Uang dan waktu lebih baik dipakai untuk CI (Bab 4) dan pipeline demo (Bab 6).

---

## Bab 6 — Vercel Free: "ya, untuk demo" — dengan syarat

### 6.1 Untuk apa Vercel di sini

Vercel = layanan hosting yang otomatis "menerbitkan" hasil karya setiap kali ada perubahan di GitHub. Untuk SDB, kegunaan utamanya adalah:

1. **Demo ke konsumen/klien**: satu tautan yang bisa dibuka di HP klien (mis. `nama-app.vercel.app`).

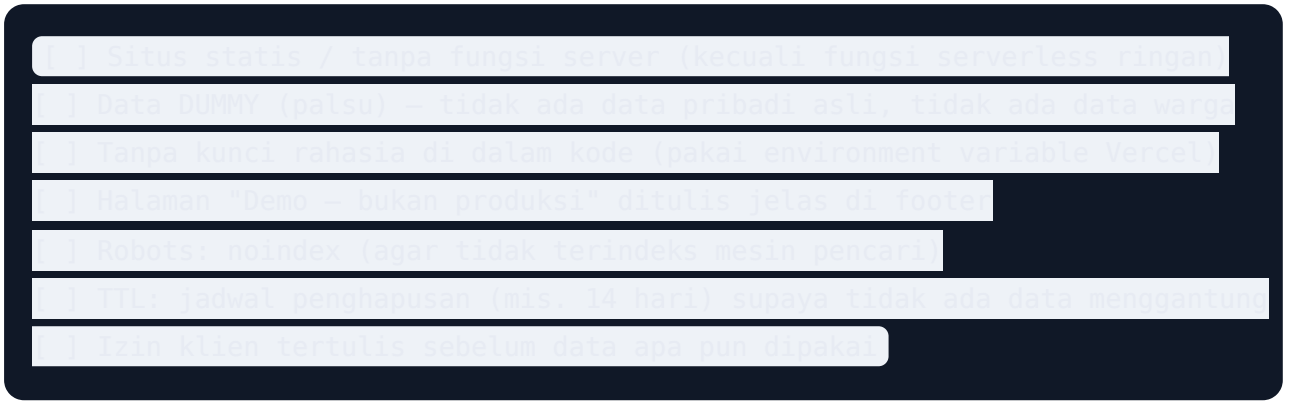
2. **Versi pratinjau (preview)**: setiap perubahan menghasilkan tautan uji tersendiri — klien bisa membandingkan versi A dan B.
3. **Bukti kerja**: tautan hidup lebih meyakinkan daripada tangkapan layar.

### 6.2 Batas paket gratis (yang perlu Anda tahu)

| Hal | Paket Hobby (gratis) |
|---|---|
| Biaya | Rp0 |
| Penggunaan | **hanya untuk non-komersial** — klien yang membayar masuk kategori komersial |
| Bandwidth | ada kuota bulanan; lewat kuota = halaman berhenti |
| Log fungsi | tersimpan singkat |
| Domain khusus | bisa, tapi fitur tim terbatas |

**Penting (jujur apa adanya)**: model komersial yang kita rancang (menjual aplikasi jadi ke klien) **tidak sepenuhnya cocok** dengan syarat "non-komersial" paket Hobby. Untuk pekerjaan yang dibayar, Vercel mengharuskan paket berbayar (Pro). Jadi Vercel Free cocok untuk: **demo, portofolio, pratinjau internal** — bukan untuk menerbitkan aplikasi berbayar milik klien secara permanen.

### 6.3 Aturan bundel untuk setiap demo (Checklist Demo)




### 6.4 Alternatif bila Vercel tidak cocok

| Alternatif | Kelebihan | Kekurangan |
|---|---|---|
| GitHub Pages | gratis, tanpa batas komersial seketat Vercel | hanya statis, tanpa fungsi server |
| Cloudflare Pages | paket gratis lebih lapang, komersial lebih ramah | aturan ToS tetap perlu dibaca |
| Server sendiri (VPS) | kontrol penuh | Anda yang jaga keamanan & pembaruan |

**Kesimpulan Bab 6**: **ya, pasang saat mulai demo** — tetapi posisikan sebagai **ruang pamer**, bukan produksi. Untuk klien berbayar, siapkan anggaran paket Pro atau pilih Cloudflare Pages/GitHub Pages.

---

## Bab 7 — Supabase: "belum perlu", kecuali aplikasi butuh database

### 7.1 Supabase itu apa (analogi)

Supabase = **gudang data + petugas administrasi + penjaga pintu**, semuanya otomatis:

| Bagian | Padanan mudah | Kegunaan |
|---|---|---|
| Postgres (database) | lemari arsip besar | menyimpan data aplikasi |
| Auth (login) | petugas penjaga pintu | pendaftaran & masuk pengguna |
| Storage | rak berkas | menyimpan gambar/dokumen |
| Realtime | pengeras suara | perubahan data langsung terlihat di layar pengguna |
| Row Level Security (RLS) | aturan "siapa boleh buka laci mana" | **wajib** untuk keamanan data |

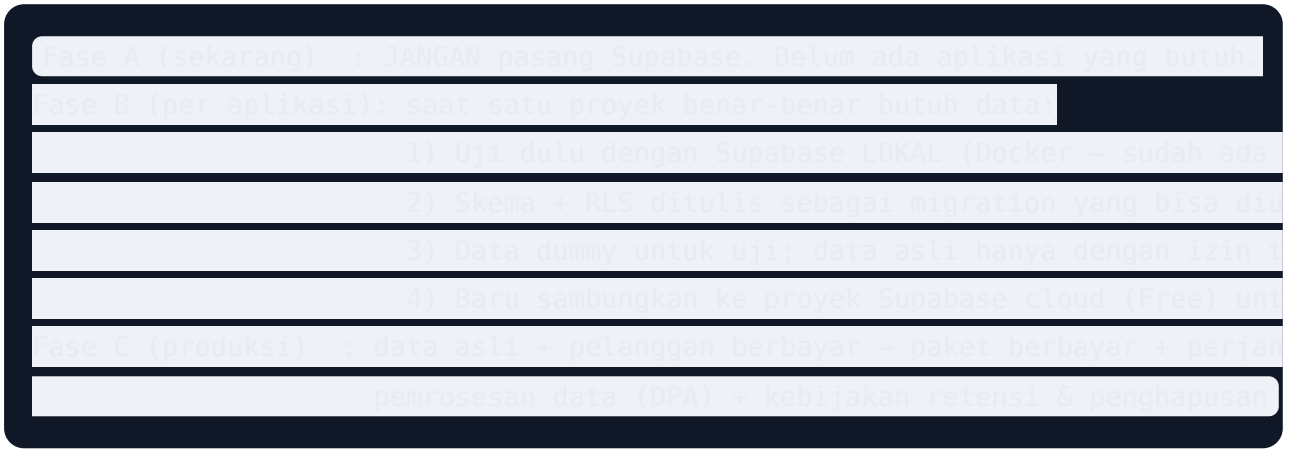
### 7.2 Kapasitas paket gratis (untuk perencanaan)

| Hal | Paket Free |
|---|---|
| Biaya | Rp0 |
| Ukuran database | 500 MB |
| Pengguna aktif bulanan | 50.000 |
| Proyek | 2 proyek aktif |
| **Proyek nganggur** | **dihentikan sementara (paused) setelah ~1 minggu tanpa aktivitas** |
| Backup | terbatas (titik pemulihan terbatas) |

**Risiko nyata yang harus dijawab, bukan diabaikan:**

| Risiko | Jawaban |
|---|---|
| Demo mati sendiri saat tidak dipakai | Untuk demo sekali jalan → **tidak masalah**. Untuk demo panjang → jadwalkan "ping" rutin, atau pakai Docker lokal saat uji |
| Data pribadi klien/warga di cloud | **Jangan** taruh data asli di paket gratis tanpa perjanjian pemrosesan data. Gunakan data palsu |
| Kunci API Supabase bocor | Kunci `anon` aman bila RLS aktif; kunci `service_role` **tidak pernah** ditaruh di frontend |
| SQL migration destruktif | Ikuti aturan repo: backup dulu, uji migration, tidak ada operasi destruktif tanpa persetujuan |

### 7.3 Jalur bertahap yang saya sarankan



**Kesimpulan Bab 7**: Supabase **bukan kebutuhan ekosistem SDB**, melainkan **kebutuhan per aplikasi**. Memasangnya sekarang = menambah risiko (data, rahasia, biaya) sebelum ada yang memakainya. Tunda sampai ada aplikasi yang benar-benar membutuhkannya — dan saat itu tiba, mulai dari Docker lokal.

---

## Bab 8 — Tabel keputusan akhir

| Layanan | Putusan | Alasan utama | Syarat sebelum dipakai |
|---|---|---|---|
| GitHub | **Pasang sekarang** | Riwayat aman + CI otomatis + Pages | Rotasi kunci yang pernah bocor; repo privat dulu |
| GitHub Desktop | **Lewati** | Sudah ada `git` + `gh`; kerja lewat agen/terminal | Tinjau manual via web GitHub |
| Vercel Free | **Pasang saat fase demo** | Ruang pamer ke klien dalam 1 tautan | Data dummy; tanda "non-komersial"; TTL; izin klien |
| Supabase | **Tunda** (per aplikasi) | Belum ada aplikasi yang butuh database | Mulai dari Docker lokal; RLS aktif; data palsu dulu |

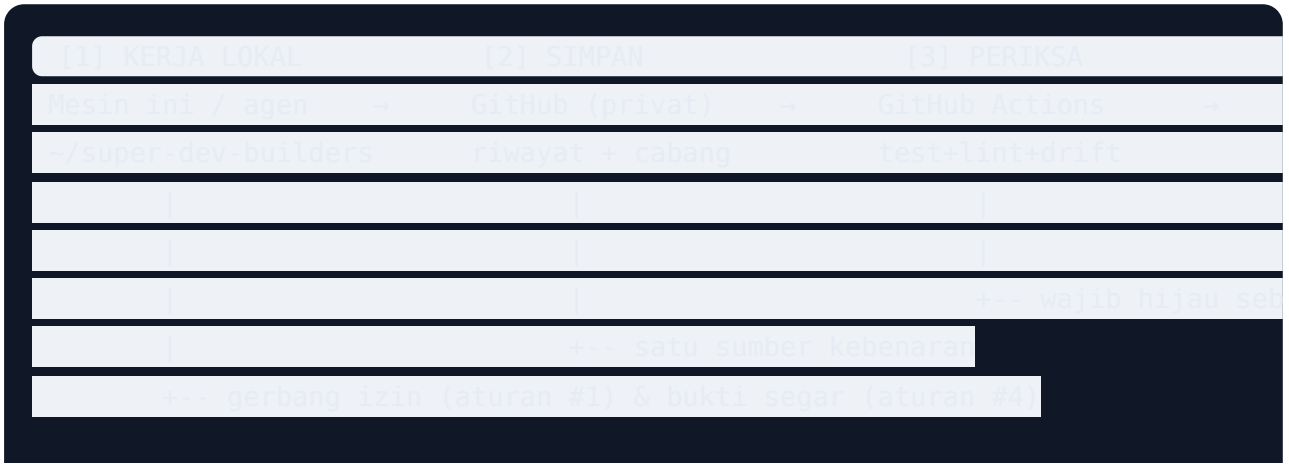
Skor kebutuhan, dari yang paling mendesak:




---

## Bab 9 — Arsitektur alur kerja (yang saya sarankan)

Alur dari ide sampai klien melihat hasilnya:



```
[5] DATA (hanya bila perlu)  →  Supabase (atau Docker lokal saat uji)
```

Aturan pembatas di setiap panah:

| Panah | Aturan yang berlaku | Konsekuensi bila dilanggar |
|---|---|---|
| 1 → 2 | Tidak ada rahasia di repo | Kunci bocor = harus rotasi segera |
| 2 → 3 | Bukti segar sebelum klaim lulus | Klaim palsu = kepercayaan hilang |
| 3 → 4 | Deploy produksi = approval manusia | Aturan #5 dilanggar |
| 4 → 5 | Data pribadi tidak masuk demo | Pelanggaran privasi + risiko hukum |

---

## Bab 10 — Menjawab pertanyaan yang belum ditanyakan (tapi penting)

**"Kalau nanti kami jual aplikasi hasil vibecoding, di mana datanya?"** Tergantung aplikasi. Tiga pilihan: (a) tanpa database (statis) — paling aman, (b) database terkelola (Supabase) — cepat, ada biaya & tanggung jawab data, (c) server sendiri — kontrol penuh, tanggung jawab penuh. Untuk tahap awal, **(a) dan (b) dengan data dummy** adalah pilihan paling bijak.

**"Apakah aman menaruh karya di cloud?"** Aman **bila** aturan mainnya dipatuhi: repo privat dulu, tanpa rahasia di dalam kode, dan hanya publikasikan bagian yang memang pameran. Yang tidak aman: menaruh kunci API, data asli, atau kode klien di repo publik.

**"Apakah ini menghambat kecepatan?"** Justru sebaliknya. CI otomatis menggantikan pemeriksaan manual — 140 test yang sekarang Anda jalankan dengan tangan akan berjalan sendiri setiap perubahan. Dan tautan demo mengurangi bolak-balik "kirim file, tunggu buka, komplain".

---

## Bab 11 — Peta jalan (berfase, tanpa langkah besar sekaligus)

| Fase | Isi | Waktu perkiraan | Syarat masuk |
|---|---|---|---|
| **F0** | Login `gh` + repo privat + push pertama | hari ini (~20 menit) | rotasi kunci rahasia |
| **F1** | GitHub Actions: `make check` + `make drift` + branch protection | 1–2 hari | F0 selesai |
| **F2** | Pipeline demo: bundel demo + Vercel Hobby + checklist demo | 2–3 hari | F1 selesai |
| **F3** | Supabase lokal (Docker) untuk proyek pertama yang butuh data | saat proyek ada | ada aplikasi nyata |
| **F4** | Supabase cloud + RLS + data dummy | per aplikasi | F3 lolos uji |
| **F5** | Produksi berbayar (Vercel Pro / VPS) + DPA + retensi data | per klien berbayar | keputusan Anda |

Prinsip fase: **satu fase, satu manfaat yang bisa dibuktikan.** Tidak melompat.

---

## Bab 12 — Biaya (rupiah, perkiraan)

| Item | Fase awal | Saat ada klien berbayar |
|---|---|---|
| GitHub | Rp0 | Rp0 (privat, anggota sedikit) |
| GitHub Actions | Rp0 (kuota gratis) | Rp0–sedikit |
| Vercel | Rp0 (Hobby, non-komersial) | ~Rp300–400 rb/bulan (Pro) |
| Supabase | Rp0 (Free, 2 proyek) | ~Rp400–600 rb/bulan (Pro) |
| Domain (opsional) | ~Rp150 rb/tahun | idem |
| **Total awal** | **~Rp0** | **~Rp700 rb–1 jt/bulan + domain** |

Catatan: biaya awal nol. Yang sebenarnya "mahal" bukan langganan, tetapi **disiplin menjaga rahasia dan data** — dan itu gratis kalau prosesnya benar.

---

## Bab 13 — Laporan verifikasi (bukti nyata)

Semua klaim di dokumen ini berasal dari pemeriksaan langsung:

| Yang diperiksa | Perintah/metode | Hasil |
|---|---|---|
| git & gh tersedia | `git --version`, `gh --version` | 2.43.0 / 2.102.0 |
| gh belum login | `gh auth status` | "You are not logged into any GitHub hosts" |
| Docker | `docker --version` | 29.8.2 |
| Node/npm | `node -v`, `npm -v` | 26.7.0 / 11.19.0 |
| Chrome | `command -v google-chrome` | `/usr/bin/google-chrome` |
| GPU | `nvidia-smi` | tidak ada GPU NVIDIA |
| SDB remote | `git remote -v` | kosong (belum tersambung) |
| Kredensial di runtime terbaru | baca `facts.json` PM | PyJWT 2.15.1, tornado 6.5.10, httpx2 2.13.1 |

### 13.1 Catatan tambahan penting (koreksi yang jujur)

Saat memeriksa kerentanan, ditemukan 37 temuan keamanan (1 CRITICAL, 20 HIGH) pada environment lama `~/.hermes/hermes-agent/venv`. Pemeriksaan lanjutan memastikan hal berikut:

1. Environment itu **bukan** lingkungan runtime Hermes yang aktif. Runtime aktif dicatat di `~/.hermes/installs/8f187687540a47e8/facts.json` dan sudah memakai versi terbaru sejak 5 Oktober 2026.
2. Namun environment lama itu **tetap hidup**: sebuah proses `real_system.prod_server` milik proyek `~/rt-rw-portal` berjalan di dalamnya.
3. Karena itu, 8 paket dinaikkan ke versi perbaikan di environment tersebut. Hasil: temuan turun **37 → 3 (0 CRITICAL, 0 HIGH)**; 3 sisanya MEDIUM tanpa versi perbaikan.
4. Proses `rt-rw-portal` tidak terganggu (ia hanya memakai modul standar Python), dan runtime Hermes tidak tersentuh sama sekali.

Dokumentasi perubahan ini juga tercatat di repo SDB.

---

## Lampiran A — Perintah yang dipakai (untuk diulang kapan saja)

```
# Deteksi status autentikasi (langkah pertama setiap kerja GitHub)
git --version; gh --version; gh auth status

# Login lewat kode perangkat (tanpa browser otomatis)
curl -s -X POST -H "Accept: application/json" \
  -d "client_id=178c6fc778ccc68e1d6a&scope=repo,read:org,gist" \
  https://github.com/login/device/code
# → buka https://github.com/login/device lalu masukkan kode yang muncul

# Setelah login
gh auth status              # memastikan akun aktif
gh auth setup-git           # menyambungkan kredensial ke git

# Repo privat + push pertama
cd ~/super-dev-builders
gh repo create super-dev-builders --private --source=. --remote=origin --push

# Uji angan (dry-run) CI sebelum push workflow
make check && make drift
```

## Lampiran B — Checklist "aman sebelum menyambung cloud"

```
[ ] Tidak ada kunci API di dalam kode maupun riwayat git
[ ] .env ada di .gitignore; template .env.template tanpa nilai asli
[ ] Repo dimulai sebagai PRIVAT
[ ] Setiap layanan baru: periksa syarat paket gratis, catat batasnya
[ ] Demo memakai DATA PALSU + tanda "bukan produksi" + noindex + TTL
[ ] Deploy produksi hanya lewat persetujuan manusia
[ ] Setiap integrasi punya bukti uji (keluaran nyata, bukan klaim)
```

---

*Dokumen ini disusun sebagai kajian kelayakan, bukan pengganti keputusan pemilik risiko. Integrasi hanya dijalankan setelah persetujuan Anda.*

Dihasilkan dari `docs/INTEGRASI_GITHUB_VERCEL_SUPABASE.md` · md_to_pdf.py · tanpa layanan pihak ketiga